
Geospatial Machine Learning

Édipo Cremon
Rennan Marujo



Spatial Machine Learning

1. Classic ML+non-spatial variables+random cross-validation,
2. Classic ML+spatial all variables+random cross-validation,
3. Classic ML+spatial all variables+spatial cross-validation,
4. Classic ML+spatial selected variables+spatial cross-validation,
5. Spatial ML+spatial selected variables+spatial cross-validation.

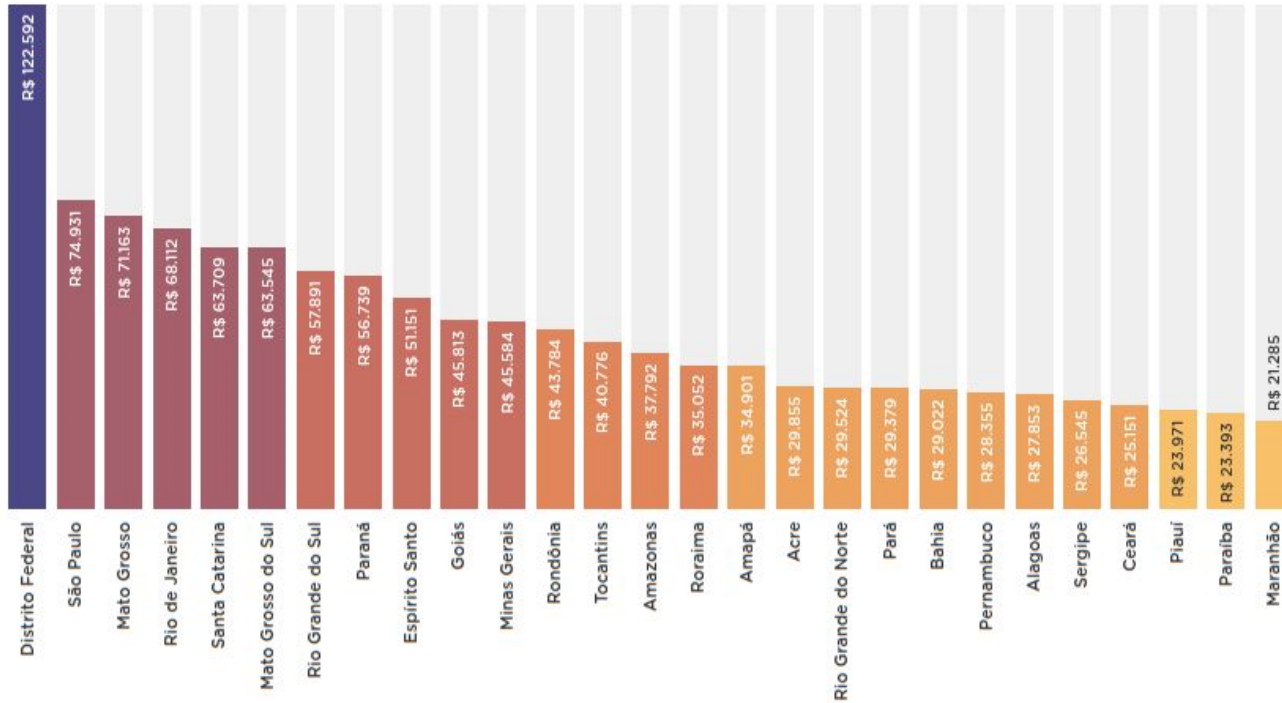


source: Spatial machine learning: new opportunities for regional science



Spatial inequality

Spatial inequality : PIB (GPD) per capita by location



Spatial inequality

Nível: Métrica:

Distribuição do PIB em 2023

Por estado, em bilhões de reais

Até 1 bi	1 a 5 bi	5 a 10 bi	10 a 50 bi
50 a 100 bi	100 a 200 bi	200 a 500 bi	Mais de 500 bi

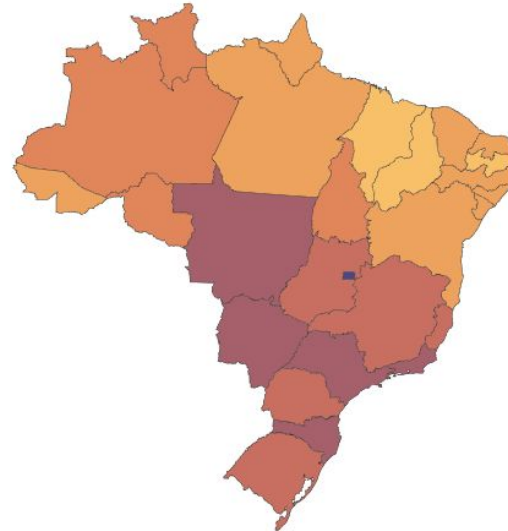


Nível: Métrica:

Distribuição do PIB per capita em 2023

Por estado, em reais

Até 15 mil	15 a 25 mil	25 a 35 mil	35 a 45 mil
45 a 60 mil	60 a 80 mil	80 a 100 mil	Mais de 100 mil



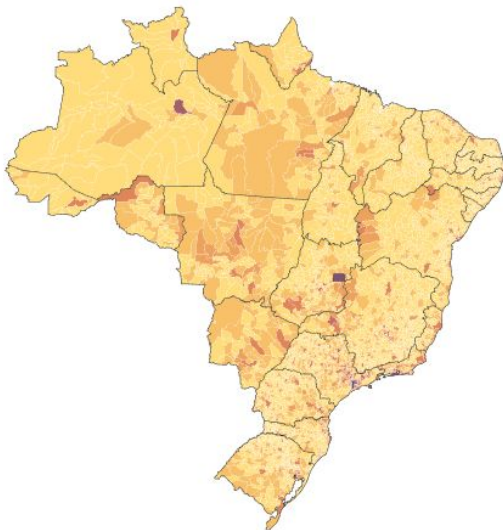
Spatial inequality

Nível: **Municípios** Métrica: **PIB total**

Distribuição do PIB em 2023

Por município, em bilhões de reais

Até 1 bi	1 a 5 bi	5 a 10 bi	10 a 50 bi
50 a 100 bi	100 a 200 bi	200 a 500 bi	Mais de 500 bi

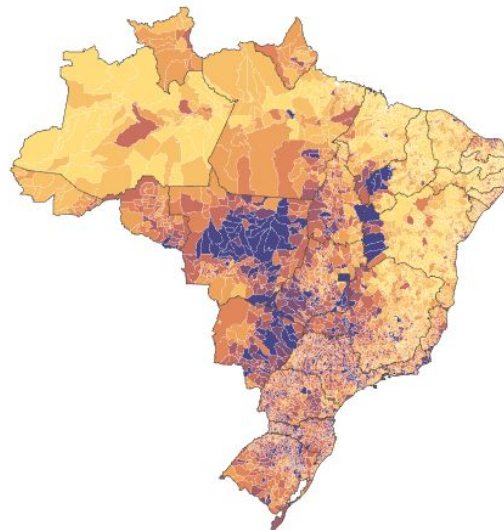


Nível: **Municípios** Métrica: **PIB per capita**

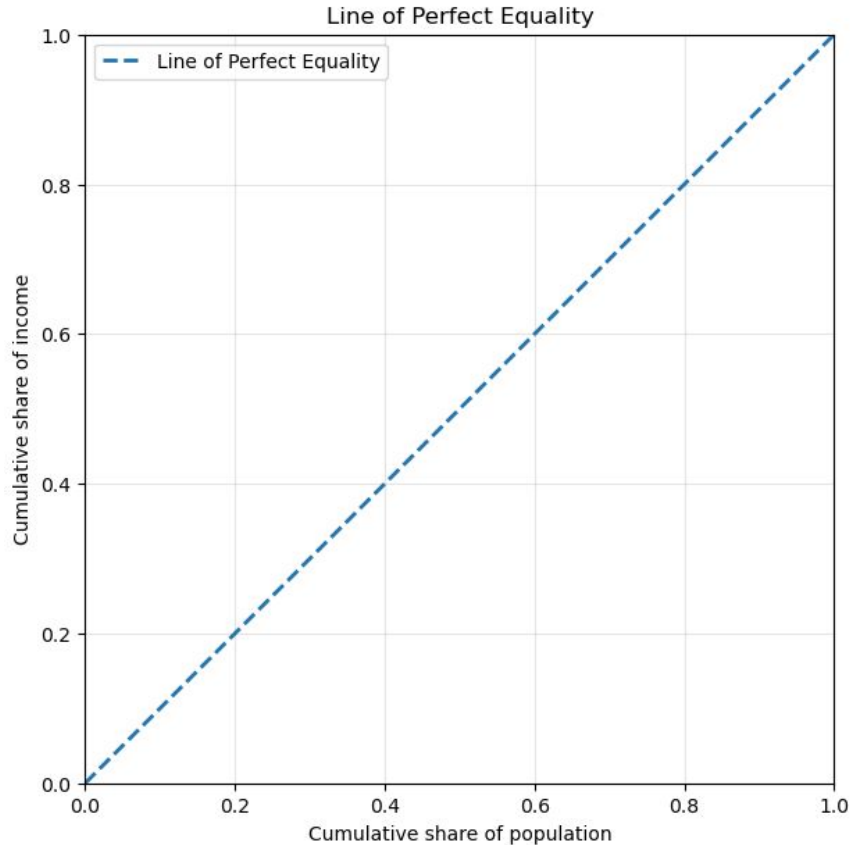
Distribuição do PIB per capita em 2023

Por município, em reais

Até 15 mil	15 a 25 mil	25 a 35 mil	35 a 45 mil
45 a 60 mil	60 a 80 mil	80 a 100 mil	Mais de 100 mil



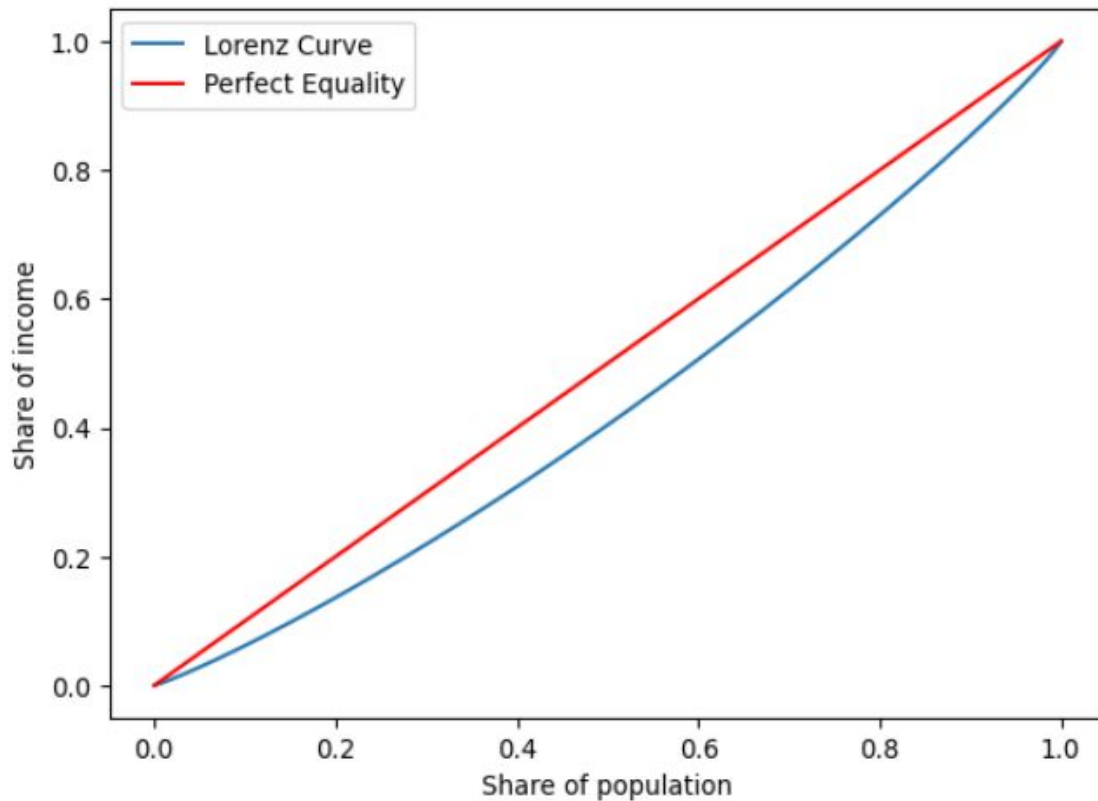
Gini index



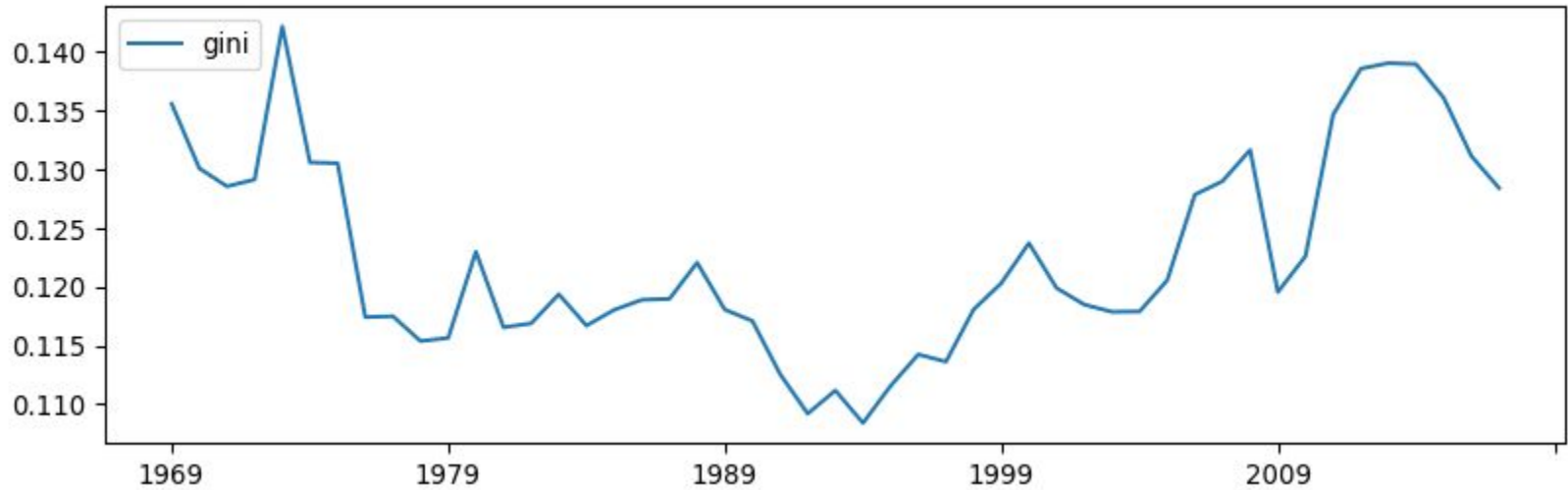
Moran -> Are similar values spatially clustered?

Gini -> How unequal is the distribution of values?

Gini index



Gini index



Spatializing Gini index

$$G = \frac{\sum_i \sum_j |y_i - y_j|}{2n^2 \bar{y}}$$

$$G = \frac{\sum_i \sum_j w_{i,j} |x_i - x_j|}{2n^2 \bar{x}} + \frac{\sum_i \sum_j (1 - w_{i,j}) |x_i - x_j|}{2n^2 \bar{x}}$$

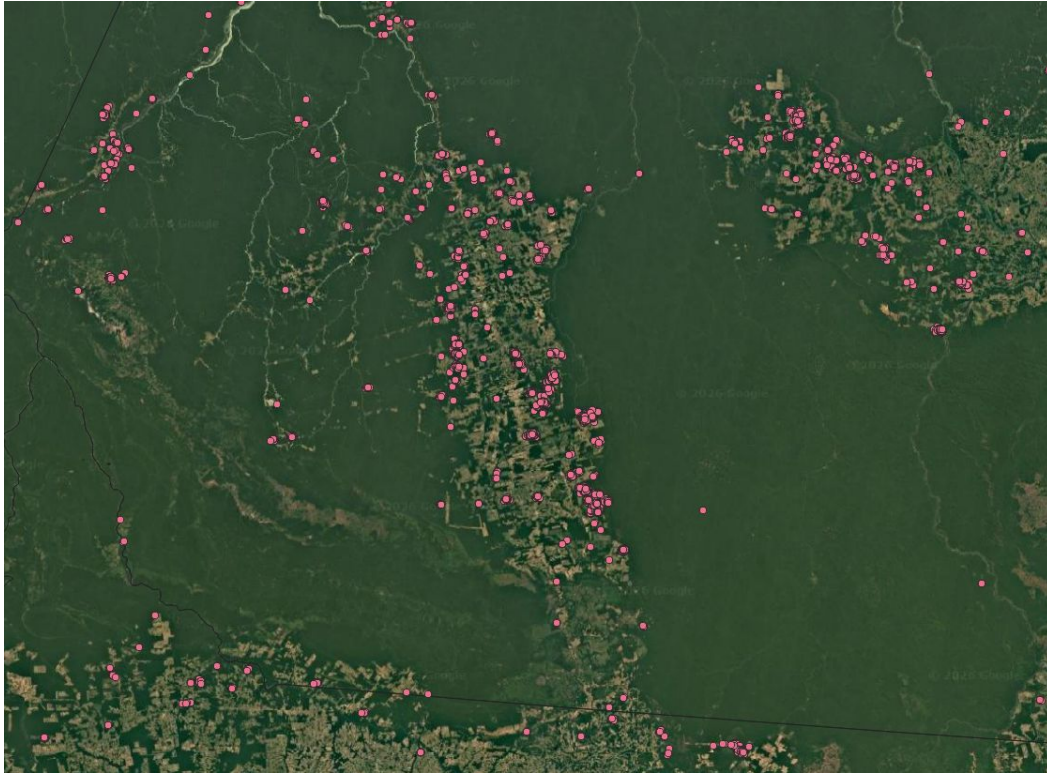
Gini -> How unequal is the distribution of values?

Spatial Gini -> How unequal are the values among spatially related observations?

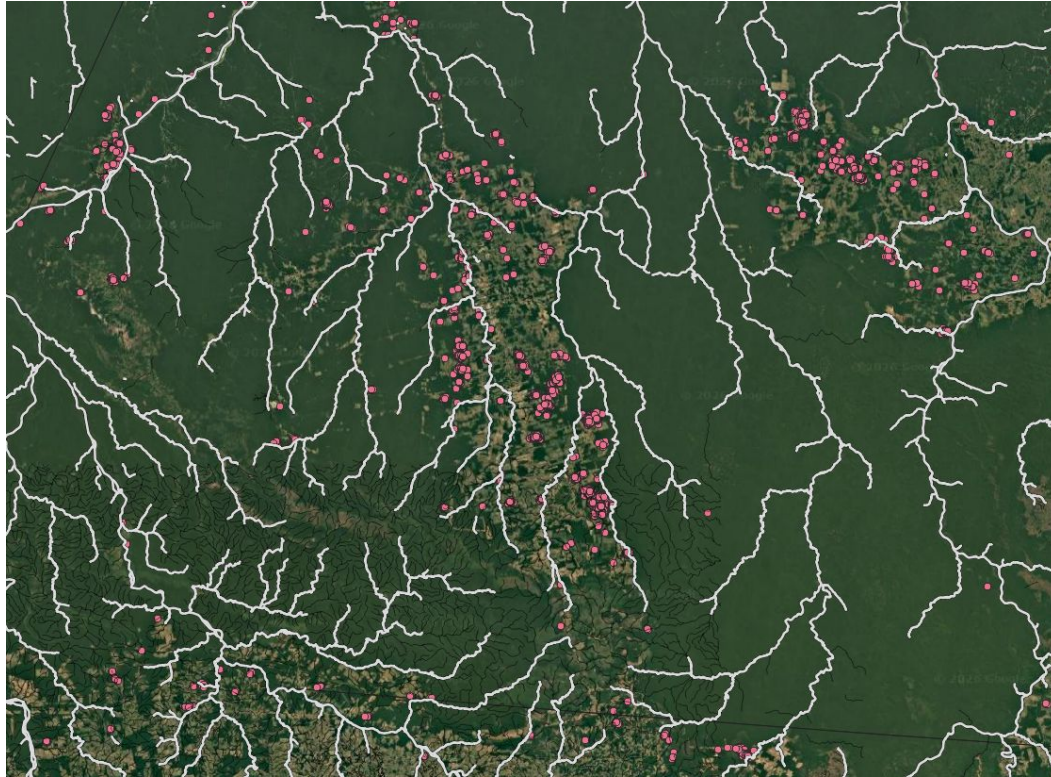


Spatial Feature Engineering

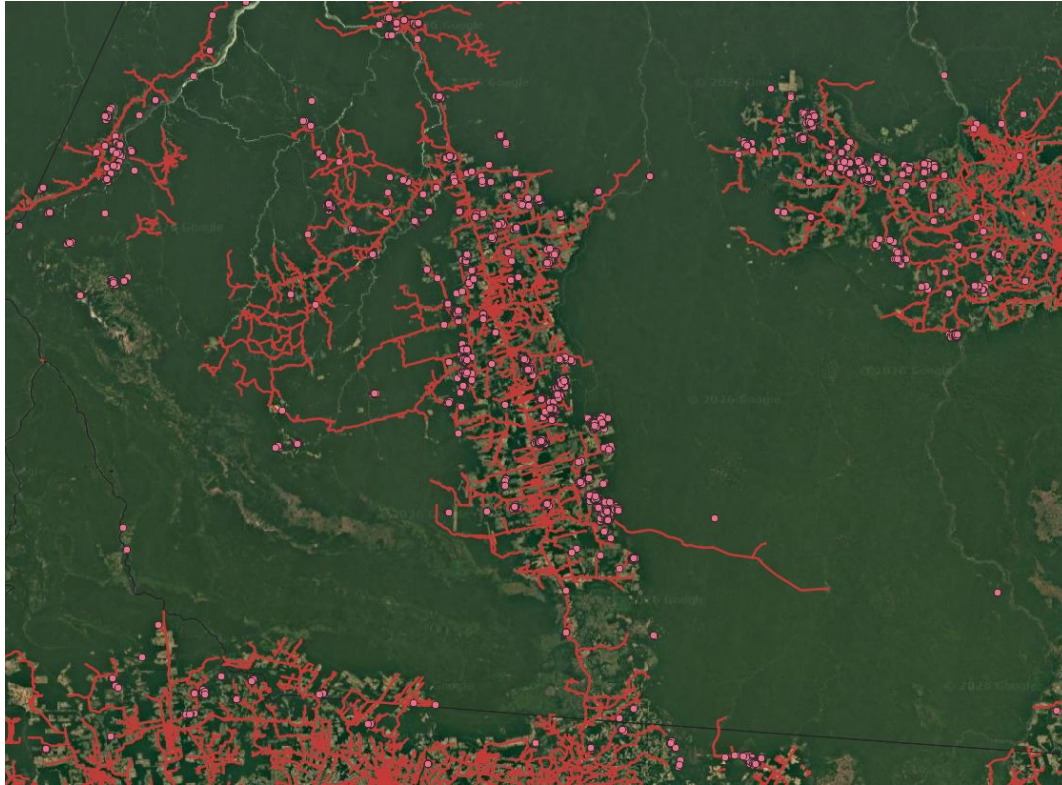
Spatial Feature Engineering: Distance based feature



Spatial Feature Engineering: Distance based feature



Spatial Feature Engineering: Distance based feature

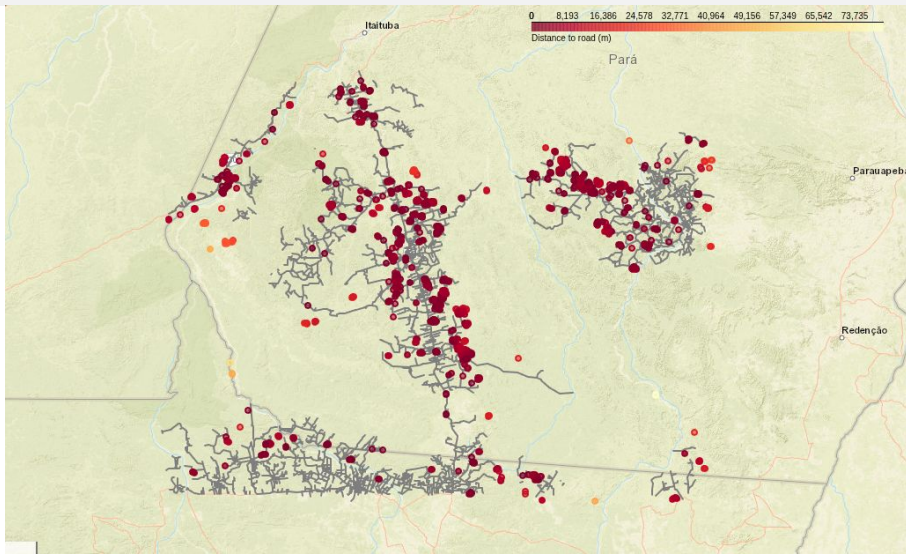


Spatial Feature Engineering: Distance based feature

```
import geopandas as gpd

fires = gpd.read_file("bdqueimadas_2026-10-01_2026-10-02.geojson").to_crs(5880)
roads = gpd.read_file("roads.gpkg").to_crs(5880)

fires = gpd.sjoin_nearest(fires, roads[["geometry"]], distance_col="dist_to_road")
fires = fires[~fires.index.duplicated()].drop(columns="index_right")
fires[["Município", "dist_to_road"]].head()
```



Spatial Feature Engineering: Counting nearby features



Spatial Feature Engineering:

Counting nearby features

```
import geopandas as gpd
```

```
gpkg_path = '/home/jovyan/geo-data-science/GBIF_Araucaria_angustifolia.gpkg'  
trees = gpd.read_file(gpkg_path).to_crs(epsg=5880)  
trees = trees[["key", "year", "stateProvince", "geometry"]]  
print(len(trees))  
trees.head(5)
```

4648

	key	year	stateProvince	geometry
0	5938070397	2026.0	Santa Catarina	POINT (5431299.943 6886309.944)
1	5938345686	2026.0	Santa Catarina	POINT (5396126.236 6863682.078)
2	5938381564	2026.0	Rio Grande do Sul	POINT (5167816.858 6745712.218)
3	5938442452	2026.0	Paraná	POINT (5505139.92 7181554.525)
4	5938507472	2026.0	Paraná	POINT (5098229.079 7322533.377)

Spatial Feature Engineering:

Counting nearby features

```
buffer_size = 5_000 # meters

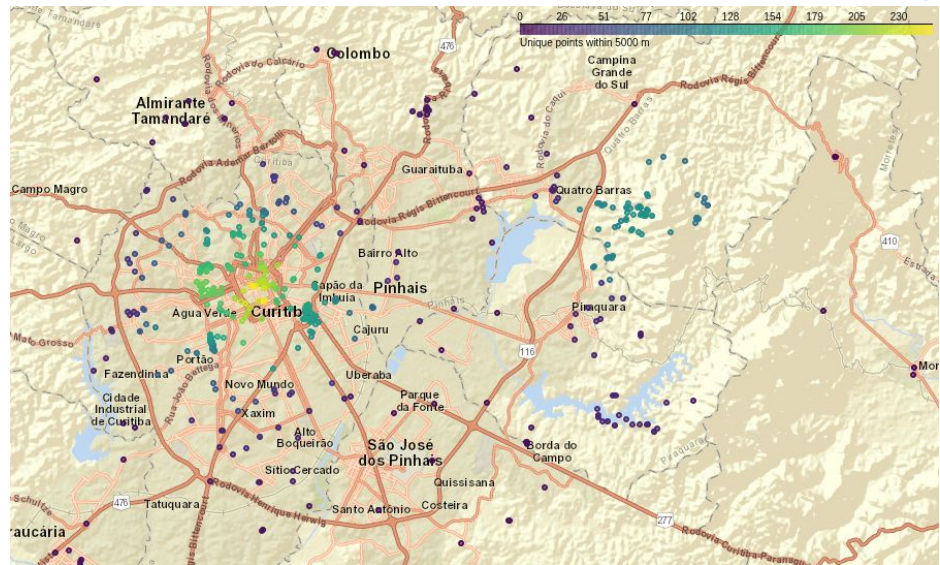
buffers = trees.copy()
buffers["geometry"] = trees.buffer(buffer_size)

# Spatial join: one row for each (buffer, tree inside it) pair
pairs = gpd.sjoin(buffers, trees, predicate="contains")

# Count pairs per buffer and subtract 1 (the tree itself)
trees["nearby_counting"] = pairs.groupby(level=0).size() - 1
trees["nearby_counting"].describe()
```

```
count      4648.000000
mean        65.053787
std         98.476661
min          0.000000
25%          4.000000
50%         29.000000
75%         75.000000
max        352.000000
Name: nearby_counting, dtype: float64
```

Spatial Feature Engineering: Counting nearby features



Spatial Feature Engineering: Point interpolation



Spatial Feature Engineering:

Point interpolation

```
import geopandas as gpd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.neighbors import KNeighborsRegressor

gpkg_path = '/home/jovyan/c3_soildata_psd_modeling_meters.gpkg'

gdf = gpd.read_file(gpkg_path) # CRS: EPSG:5880 (meters)
topsoil = gdf.sort_values("profundidade").drop_duplicates("id")
topsoil = topsoil.rename(columns={"argila": "clay"})[["id", "clay", "geometry"]]
print(gdf.crs, len(gdf), len(topsoil))
topsoil.head()
```

		id	clay	geometry
47749	ctb0779-PERFIL-8-RJ	80	POINT (6470784.171 7839869.377)	
46901	ctb0769-87	20	POINT (6221894.968 7424584.283)	
32039	ctb0568-Perfil-96	20	POINT (6221848.155 7424530.897)	
8249	ctb0033-RO1002	270	POINT (3887035.038 8613337.654)	
13726	ctb0033-RO2012	140	POINT (3813626.5 8795864.748)	

Spatial Feature Engineering:

Point interpolation

```
X = np.column_stack([topsoil.geometry.x, topsoil.geometry.y])
y = topsoil["clay"].values

model = KNeighborsRegressor(n_neighbors=10, weights="distance")
model.fit(X, y)

cell_size = 10_000
xmin, ymin, xmax, ymax = topsoil.total_bounds
grid_x, grid_y = np.meshgrid(np.arange(xmin, xmax, cell_size), np.arange(ymin, ymax, cell_size))
grid_points = np.column_stack([grid_x.ravel(), grid_y.ravel()])

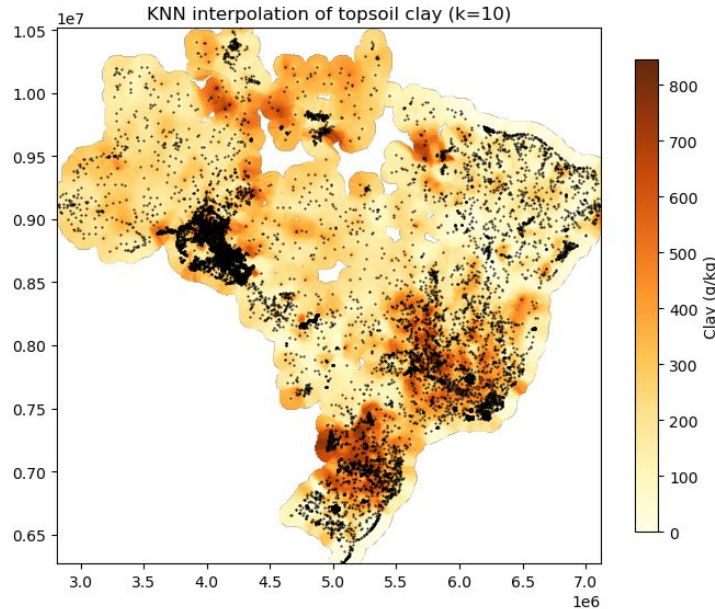
clay_pred = model.predict(grid_points).reshape(grid_x.shape)

# Mask cells farther than 100 km from any sample (avoid extrapolation)
dist, _ = model.kneighbors(grid_points, n_neighbors=1)
clay_pred[dist.reshape(grid_x.shape) > 100_000] = np.nan
```

Spatial Feature Engineering:

Point interpolation

```
fig, ax = plt.subplots(figsize=(8, 8))
img = ax.imshow(clay_pred, origin="lower", extent=(xmin, xmax, ymin, ymax), cmap="YlOrBr")
topsoil.plot(ax=ax, color="k", markersize=0.2)
fig.colorbar(img, ax=ax, shrink=0.7, label="Clay (g/kg)")
ax.set_title("KNN interpolation of topsoil clay (k=10)");
```





Spatial Sampling

Spatial Sampling

Spatial sampling with R

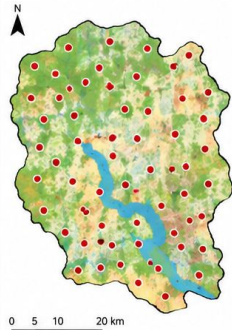
Dick J. Brus

2023-09-03

<https://dickbrus.github.io/SpatialSamplingwithR/GeneralIntro.html>

Spatial Sampling

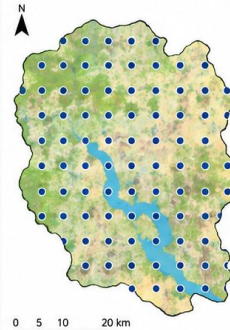
Random sampling



- Sample points
- Study area

- Selects locations completely by chance across your study area.
- Works well when you have little prior knowledge about spatial patterns and want to avoid bias.
- Provides statistically robust results but might miss important features or create inefficient travel routes for data collection.

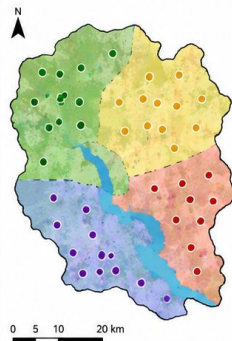
Systematic sampling



- Sample points
- Study area

- Places sample points at regular intervals across a grid pattern.
- Ensures even coverage of your study area and often captures spatial trends effectively.
- Useful for environmental monitoring or infrastructure assessments where uniform coverage matters more than statistical randomness.

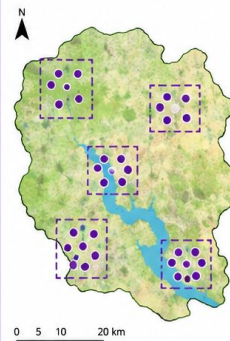
Stratified sampling



- Stratum 1 (forest)
- Stratum 2 (savanna)
- Stratum 3 (agriculture)
- Stratum 4 (urban)
- Sample points
- Study area

- Divides your study area into distinct zones or strata based on known characteristics, then samples within each zone.
- Guarantees representation of different geographic conditions and often provides more precise estimates than simple random sampling.
- Works well when your area contains diverse environments or land uses.

Cluster sampling

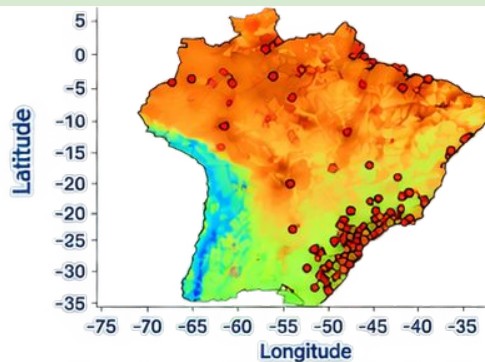


- Sample points (cluster)
- Cluster area
- Study area

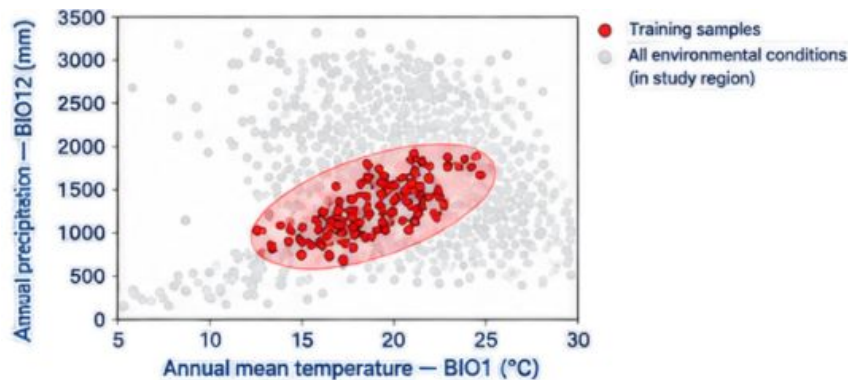
- Groups nearby locations together and samples entire clusters rather than individual points.
- Reduces travel costs and time while maintaining statistical validity.
- Ideal for large-scale studies or when accessing individual points proves difficult.

Geographic coverage $\neq$ Feature-space coverage

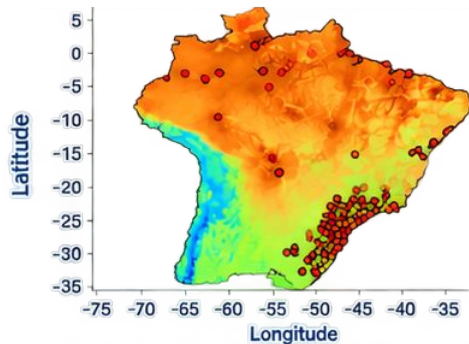
Good geographic coverage



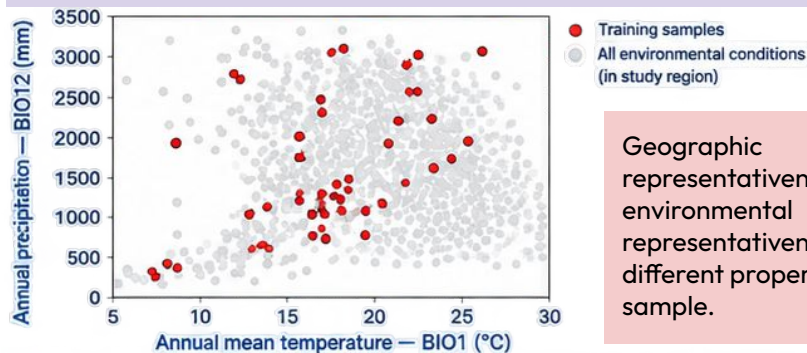
Limited feature-space coverage



Limited geographic coverage



Broader feature-space coverage

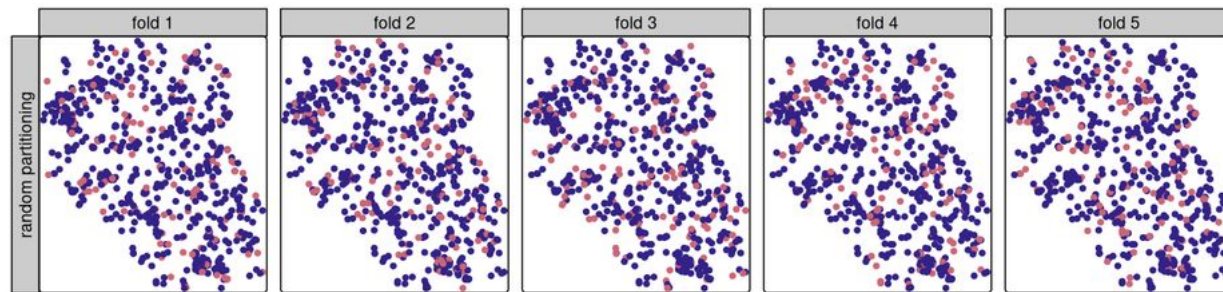
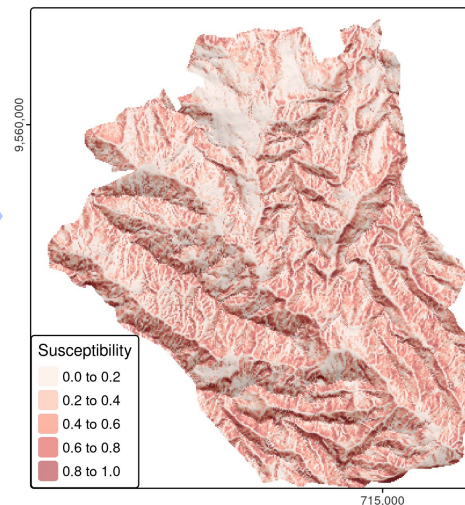
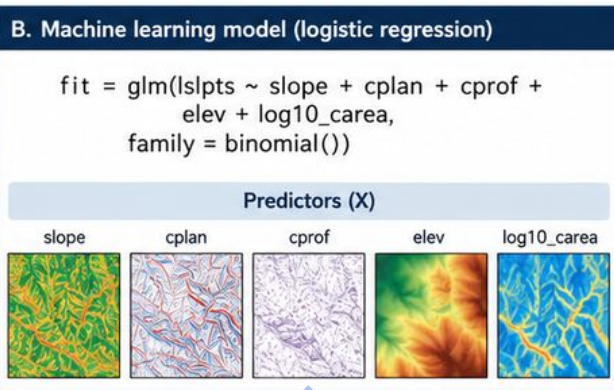
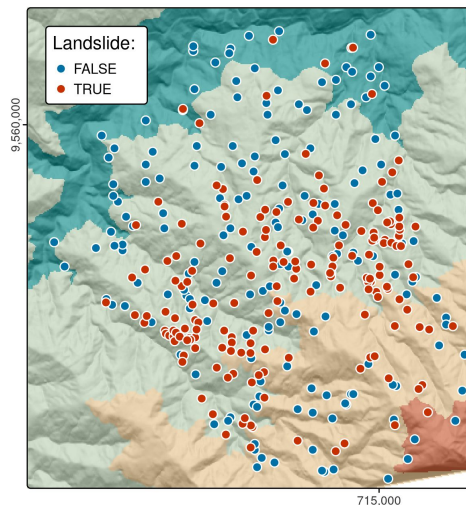


Geographic representativeness and environmental representativeness are different properties of a sample.

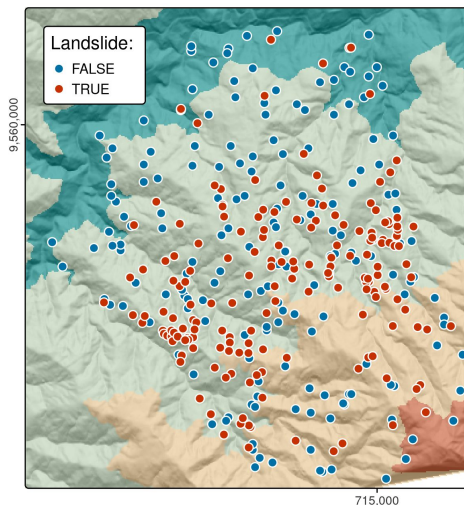


Spatial cross-validation & Evaluation

Why random cross-validation can mislead



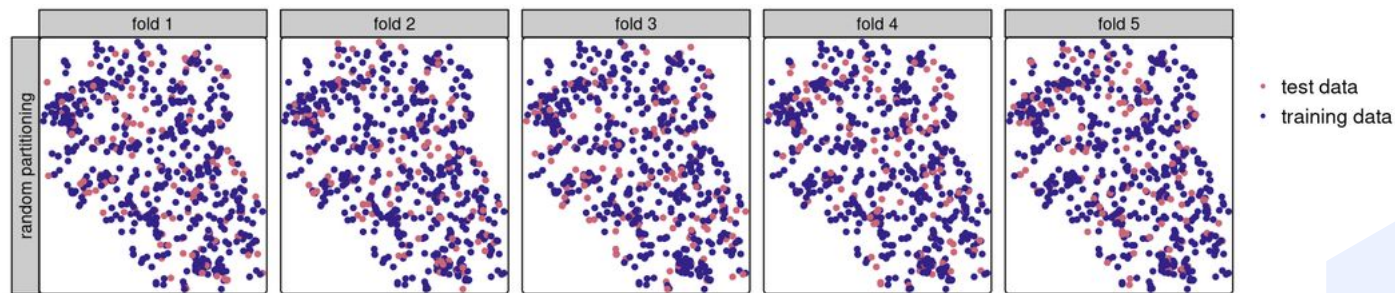
Why random cross-validation can mislead



Spatial dependence breaks random splitting

- Nearby observations may be similar
- Random CV mixes nearby train/test points
- Test data are not truly independent
- Performance can look too optimistic

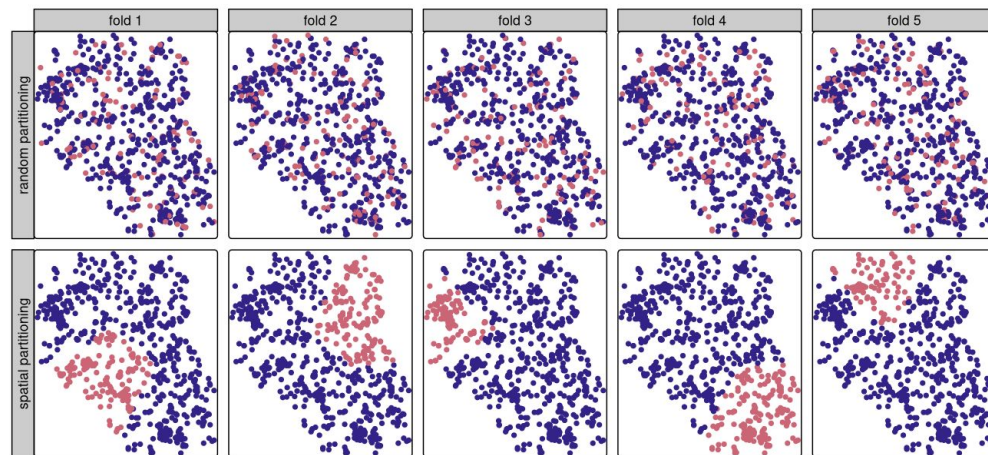
High accuracy can reflect spatial leakage, not spatial generalization.



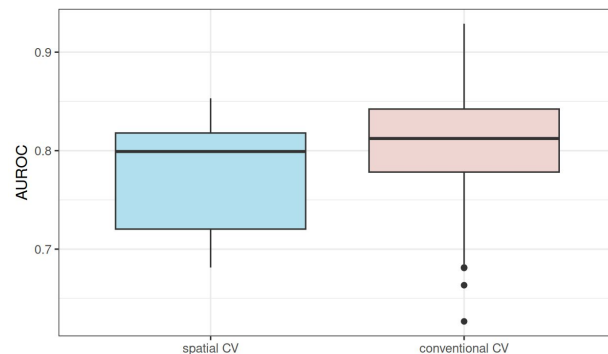
Spatial cross-validation

Separate observations in geographic space

- Training and validation points are spatially separated
- Spatial CV should mimic the intended prediction scenario
- Performance estimates become less optimistic



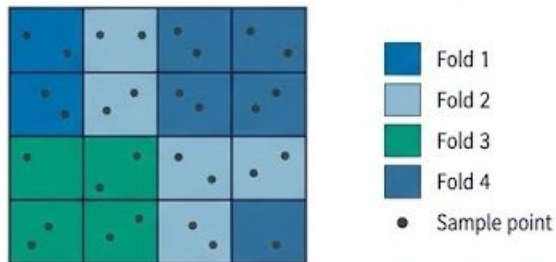
- test data
- training data



Common spatial resampling strategies

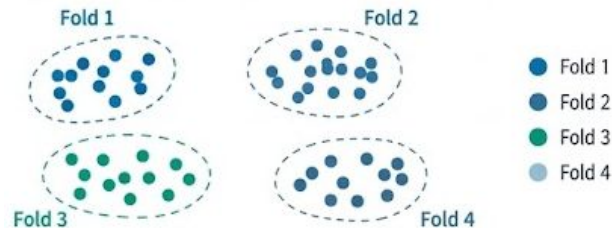
Spatial blocks

Contiguous geographic regions as folds



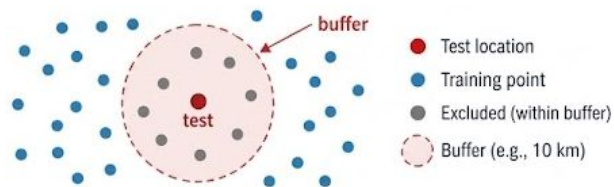
Spatial clusters

Nearby locations grouped into folds



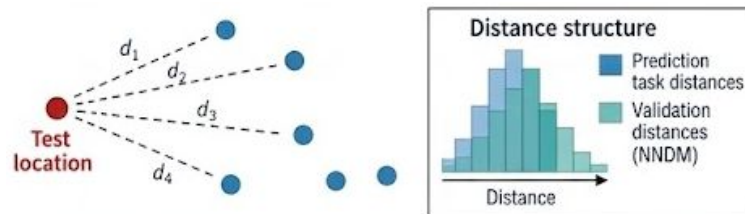
Buffered / Spatial LOO (leave-one-out)

Exclude neighbors around test locations



kNNNDM / distance matching

Match train-test distances to the prediction domain



kNNNDM creates folds so that train-test distances resemble the distances expected in the prediction domain.

Do we still need an independent test set?

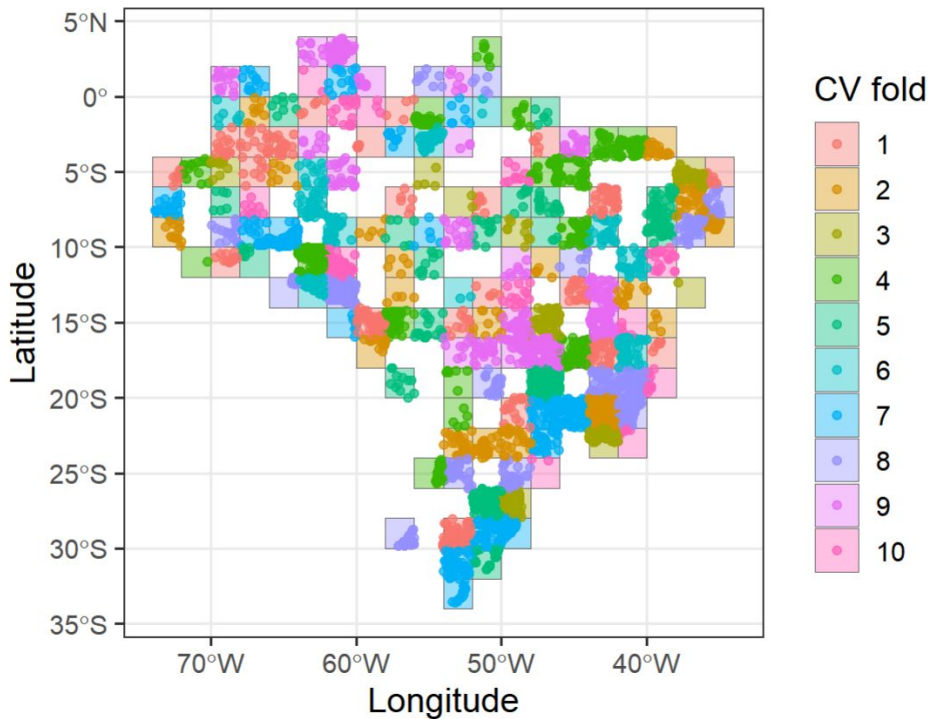
Option 1 – Spatial cross-validation only

Spatial CV

- All samples contribute to model assessment.
- Efficient when sample size is limited.
- Spatial folds reduce train–test spatial dependence.
- Common approach in geospatial ML.

Spatial cross-validation

10 folds based on $2^\circ \times 2^\circ$ geographic blocks



Do we still need an independent test set?

Option 2 - Spatial holdout + spatial CV

Spatial CV

Samples → spatial blocks → 70% training ,
30% independent spatial test

Within the 70% training set:

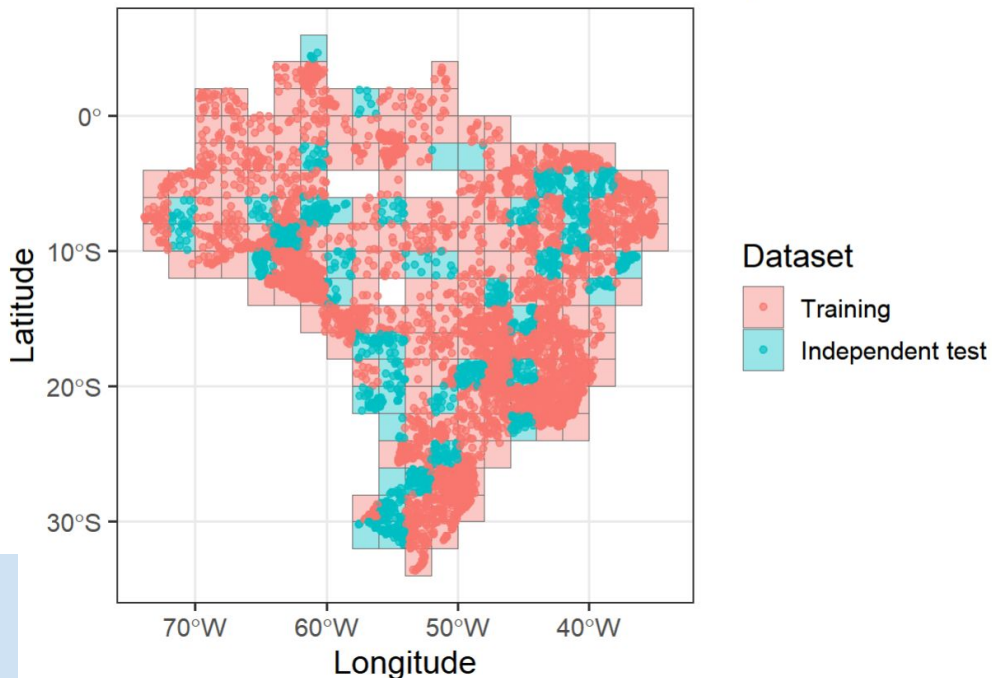
spatial blocks → K spatial folds →
hyperparameter tuning

- Spatial CV is used for **model selection and tuning**.
- The spatial holdout provides a **final, independent performance estimate**.
- Stronger separation between model development and final evaluation.
- Costs data: fewer samples are available for training and tuning.

Use **spatial CV + independent spatial test**
when enough data are available and a strong
final generalization test is required.

Spatial training and independent test sets

Complete $2^\circ \times 2^\circ$ blocks are assigned to only one dataset



Validation design must match the prediction task

How we split the data determines what “model performance” actually means.

1. Define the prediction task

Prediction within well-sampled regions

→ weaker spatial separation may be acceptable

Prediction to new geographic regions

→ stronger spatial separation / spatial holdout

Environmental extrapolation

→ geographic separation alone may not be enough

2. Separate model development from final evaluation (When enough data are available)

Training data

→ spatial CV

→ hyperparameter tuning + model selection

Independent spatial test set

→ keep untouched during model development

→ use once for final performance estimation

Validation should reproduce the intended prediction scenario, not just partition the available data.

Geospatial Machine Learning

Édipo Cremon
Rennan Marujo

